

4-dars

Model Architecture'dan Training and Optimization'ga o'tish

Computer Vision modelida avval modelning tuzilishi, ya'ni architecture tushuniladi. Architecture — model qanday qismlardan iborat bo'lishi, rasm modelga qayerdan kirishi, qaysi qatlamlarda qayta ishlanishi va qayerda yakuniy natija chiqishini bildiradigan tuzilma hisoblanadi.

Architecture so'zining o'zbekcha izohi — “model tuzilishi”, “model arxitekturası”, “model ichki qurilishi”. Model architecture bor bo'lgani bilan model o'z-o'zidan o'rganib qolmaydi. Architecture modelning tanasi yoki skeleti bo'lsa, training va optimization shu modelning o'rganish jarayoni hisoblanadi.

Training — modelni data orqali o'qitish jarayoni. Data — ma'lumot. Computer Visionda data ko'pincha rasm yoki video bo'ladi. Image Classification vazifasida data rasm va uning labelidan iborat bo'ladi. Label — rasmning to'g'ri javobi yoki class nomi. Masalan, mushuk rasmi uchun label `cat`, it rasmi uchun label `dog` bo'ladi.

Optimization — model xatosini kamaytirish uchun model ichidagi o'rganiladigan qiymatlarni yangilash jarayoni. Optimization o'zbekchada “optimallashtirish”, “yaxshilash”, “eng yaxshi holatga olib kelish” degan ma'noda ishlatiladi.

Model architecture umumiy tarzda quyidagicha tushuniladi:

Input Image → Backbone → Neck → Head → Output

Input Image — modelga kiradigan rasm.

Backbone — rasm ichidan kerakli featurelarni ajratib oladigan asosiy qism. Feature — rasm ichidagi foydali belgi yoki xususiyat. Masalan, mushukni tanishda quloq, koʻz, moʻylov, yuz shakli feature boʻlishi mumkin.

Neck — backbone ajratgan featurelarni tartiblaydigan, ixchamlaydigan yoki head qismiga mos formatga keltiradigan oraliq qism.

Head — yakuniy qaror chiqaradigan qism. Image Classificationda head class prediction chiqaradi.

Output — modeldan chiqqan natija. Masalan, cat, dog, horse, deer.

Training and Optimization jarayoni qoʻshilganda model faqat rasmni oʻtkazib natija chiqarib qolmaydi. U oʻz xatosini ham oʻlchaydi, xatoni orqaga qaytaradi, weightlarni yangilaydi va keyingi safar yaxshiroq ishlashga harakat qiladi.

Weight — model ichidagi oʻrganiladigan vazn. Weightlar model qaysi featurega koʻproq yoki kamroq ahamiyat berishini belgilaydi. Bias — model hisoblashiga qoʻshimcha moslashuv beradigan parametr. Weight va bias training davomida oʻzgaradi.

Training and Optimization umumiy oqimi quyidagicha boʻladi:

Input → Model → Prediction → Loss → Backpropagation
→ Optimizer Step → Better Model

Input — modelga kiruvchi rasm yoki batch.

Model — neural network.

Prediction — model taxmini.

Loss — prediction qanchalik xato ekanini ko‘rsatadigan qiymat.

Backpropagation — xatoni orqaga tarqatish jarayoni.

Optimizer Step — optimizer weightlarni yangilaydigan qadam.

Better Model — oldingisiga qaraganda yaxshiroq o‘rgangan model.

Neural network — neyron tarmoq. Bu ma’lumotlardan pattern o‘rganadigan model hisoblanadi. Pattern — takrorlanadigan belgi, qonuniyat yoki ma’no beruvchi tuzilma.

Image Classification system architecture

Image Classification system architecture chizmasi misol sifatida olinsa, model yaratish jarayoni 4 katta phase, ya’ni bosqichga ajratiladi. Phase — bosqich degani.

Birinchi bosqich — Data Preparation. Data Preparation — ma’lumotni tayyorlash. Bu qismda data acquisition, preprocessing va augmentation bajariladi.

Data Acquisition — rasm yig‘ish va label berish. Acquisition — yig‘ish yoki olish degani. Image Classification modelini o‘qitish uchun avval rasmlar to‘planadi va har bir rasimga to‘g‘ri class nomi beriladi. Masalan, **cat** va **dog** classification qilinsa, mushuk rasmlari **cat**, it rasmlari **dog** labeli bilan belgilanadi.

Preprocessing — rasmni modelga mos holatga keltirish. Bu yerda resize, normalize, center crop kabi amallar bajariladi. Resize — rasm o‘lchamini o‘zgartirish. Normalize — pixel qiymatlarini modelga qulay oraliqqa keltirish. Center crop — rasmning markaziy qismini kesib olish.

Augmentation — mavjud rasmlardan yangi ko‘rinishlar yaratish. Masalan, rasmni aylantirish, chap-o‘ngga ag‘darish, rangini ozgina

o'zgartirish, brightness yoki contrastni o'zgartirish. Brightness — yorug'lik. Contrast — ranglar orasidagi farq. Augmentation modelning generalization qobiliyatini yaxshilaydi. Generalization — modelning oldin ko'rmagan yangi rasmda ham yaxshi ishlashi.

Ikkinchi bosqich — Model Architecture. Model Architecture ichida backbone, neck va head qismlari bor. Backbone input layer, convolutional layers va pooling layers orqali feature extraction qiladi. Input layer — rasm modelga kiradigan qatlam. Convolutional layers — rasm ichidan qirra, chiziq, rang o'zgarishi va shakllarni ajratadigan qatlamlar. Pooling layers — muhim ma'lumotni saqlagan holda feature map hajmini kichraytiradigan qatlamlar.

Feature extraction — rasm ichidan foydali belgilarni ajratib olish. Masalan, model avval oddiy qirralarni, keyin ko'z, quloq, g'ildirak, barg shakli kabi murakkab belgilarni o'rganishi mumkin.

Neck qismida Global Average Pooling ishlatilishi ko'rsatiladi. Global Average Pooling — feature mapni ixcham vector ko'rinishiga keltiradigan amal. Feature map — model ajratgan belgilar xaritasi. Vector — raqamlar ro'yxati.

Head qismida Fully Connected Layer va Softmax Activation ishlatiladi. Fully Connected Layer — barcha kiruvchi qiymatlar chiqish neuronlari bilan bog'langan qatlam. Softmax Activation — model chiqargan scorelarni ehtimollikka aylantiradigan funksiya.

Score — model har bir class uchun chiqargan ball. Probability — ehtimollik. Masalan:

Cat: 98%

Dog: 2%

Bu natija model rasmni 98% ehtimol bilan cat deb topganini bildiradi.

Uchinchi bosqich — Training & Optimization. Bu qismda forward pass, loss calculation, backpropagation, optimizer va validation bor.

Forward Pass — rasm model ichidan oldinga qarab o‘tib prediction hosil qiladigan jarayon.

Loss Calculation — model xatosini hisoblash. Loss — model predictioni haqiqiy javobdan qanchalik uzoq ekanini bildiradi.

Backpropagation — loss asosida xatoni orqaga tarqatish va gradientlarni hisoblash. Gradient — weight qaysi yo‘nalishda va qanchalik o‘zgarishi kerakligini ko‘rsatadigan qiymat.

Optimizer — gradientlardan foydalanib weightlarni yangilaydigan algoritm.

Validation — training jarayonida modelni alohida data’da tekshirish. Validation orqali model yaxshi o‘rganyaptimi yoki overfitting qilyaptimi kuzatiladi.

To‘rtinchi bosqich — Deployment & Inference. Deployment — modelni real ishlaydigan tizimga chiqarish. Inference — train qilingan model yordamida yangi rasmga prediction olish.

Deployment Optimization qismida Quantization, ONNX va TensorRT ko‘rsatiladi. Quantization — modelni yengillashtirish uchun son formatlarini kichraytirish. ONNX — modelni turli frameworklarda ishlatishga yordam beradigan format. TensorRT — NVIDIA qurilmalarida model inference tezligini oshirishga yordam beradigan vosita.

Monitoring — productga chiqqan model ishlashini kuzatish. Monitoringda drift va performance kuzatiladi. Drift — real hayotdagi data training data’dan farq qilib ketishi. Performance — modelning ishlash sifati va tezligi.

Ushbu architecture chizmasi model yaratish faqat bitta model yozishdan iborat emasligini ko'rsatadi. To'liq jarayon data tayyorlashdan boshlanadi, model architecture orqali davom etadi, training va optimization orqali model o'rganadi, deployment va monitoring orqali real product holatida ishlaydi.

Training and Optimization nima?

Training and Optimization modelni xatolardan o'rgatish va vaqt o'tishi bilan yaxshilash jarayonidir. Bu jarayon oddiy hayotdagi o'quvchi mashq qilishiga o'xshaydi. O'quvchi avval mavzuni ko'radi, misollarni ishlaydi, xatolarini tekshiradi, feedback oladi va keyingi safar yaxshiroq ishlashga harakat qiladi.

Feedback — qayta aloqa yoki xato haqida ma'lumot. Model uchun feedback loss orqali keladi. Loss modelga prediction qanchalik noto'g'ri ekanini bildiradi.

Training — modelni ma'lumotdan o'rgatish jarayoni. Model ko'p marta rasm ko'radi, prediction qiladi, xato qiladi va shu xatodan o'rganadi.

Optimization — model parametrlarini yangilab, lossni kamaytirish va performance'ni oshirish jarayoni. Parameter — model ichidagi o'rganiladigan qiymat. Weight va bias parameter hisoblanadi.

Training loop — training jarayonining takrorlanadigan sikli. Loop — takrorlanish. Training loop quyidagi qismlardan iborat:

1. Input image
2. Model
3. Prediction
4. Loss
5. Optimizer update

6. Improved model

Input image — datasetdan kelgan rasm. Dataset — model oʻrganadigan rasmlar toʻplami.

Model — neural network. Neural network rasm ichidagi featurelarni tahlil qilib prediction chiqaradi.

Prediction — model aytgan javob. Masalan, dog, cat, bird.

Loss — prediction xatosi. Loss yuqori boʻlsa, model yomonroq prediction qilgan boʻladi. Loss past boʻlsa, model yaxshiroq prediction qilgan boʻladi.

Optimizer update — optimizer weightlarni yangilaydigan qadam. Update — yangilash.

Improved model — yangilangan, keyingi safar yaxshiroq ishlashi kutiladigan model.

Training loop chizmasida it rasmi input sifatida keladi. Model prediction sifatida quyidagi ehtimolliklarni chiqaradi:

Dog: 0.30

Cat: 0.65

Bird: 0.05

True label, ya'ni haqiqiy javob Dog. Model esa eng katta ehtimollikni Cat classiga bergan. Shuning uchun prediction notoʻgʻri hisoblanadi. Loss qiymati 0.65 sifatida koʻrsatiladi. Loss qanchalik katta boʻlsa, model xatosi shunchalik sezilarli boʻladi. Keyin optimizer weightlarni yangilaydi va model keyingi batchlarda yaxshiroq javob berishga harakat qiladi.

Bu jarayon ko‘p batch va ko‘p epoch davomida takrorlanadi. Batch — datasetning kichik qismi. Epoch — butun datasetni bir marta to‘liq ko‘rib chiqish.

Training and Optimization hayotiy o‘xshatmada o‘quvchi mashq qilishiga o‘xshaydi:

O‘quvchi mavzuni ko‘radi

↓

Misol ishlaydi

↓

Xatosini tekshiradi

↓

Feedback oladi

↓

Keyingi safar yaxshiroq ishlaydi

Modelda esa bu quyidagicha bo‘ladi:

Data kiradi

↓

Model prediction qiladi

↓

Loss xatoni o‘lchaydi

↓

Backpropagation xatoni orqaga tarqatadi

↓

Optimizer weightlarni yangilaydi

↓

Model yaxshilanadi

PyTorch training step misolida quyidagi kod ishlatiladi:


```
optimizer.zero_grad()  
outputs = model(images)  
loss = criterion(outputs, labels)  
loss.backward()  
optimizer.step()
```

`optimizer.zero_grad()` — eski gradientlarni tozalaydi.

`outputs = model(images)` — forward pass bajariladi.

`loss = criterion(outputs, labels)` — loss hisoblanadi.

`loss.backward()` — backpropagation bajariladi.

`optimizer.step()` — weightlar yangilanadi.

Criterion — loss function. Loss function — model xatosini hisoblaydigan funksiya.

Training and Optimizationdagi asosiy terminlar quyidagicha tushuniladi:

Data — model o'rganadigan misollar.

Model — prediction qiladigan weightlarga ega funksiya.

Prediction — modelning berilgan input uchun chiqargan javobi.

Loss — prediction true label bilan solishtirilganda qanchalik noto'g'ri ekanini o'lchaydi.

Optimizer — lossni kamaytirish uchun parameterlarni yangilaydigan algoritm.

Epoch — butun training datasetdan bir marta to'liq o'tish.

Training and Optimization'ning simple view va technical view ko'rinishi

Training and Optimization jarayoni ikki xil ko'rinishda tushuntiriladi: simple view va technical view.

Simple view — sodda ko'rinish. Bu yerda model o'quvchi kabi tasvirlanadi. Rasm yoki misol input sifatida keladi. Model taxmin qiladi. Keyin prediction true label bilan solishtiriladi. Agar xato bo'lsa, correction, ya'ni tuzatish bajariladi. Correction — xatoni tuzatish. Model xatosidan o'rganib, keyingi safar yaxshiroq ishlashga harakat qiladi.

Simple view oqimi:

Input → Prediction → Correction → Learn & Improve

Input qismida cat, dog yoki car kabi rasmlar beriladi. Prediction qismida model guess qiladi. Guess — taxmin. Masalan, model itni dog deb aytishi yoki adashib cat deb aytishi mumkin. Correction qismida true label bilan solishtirish bor. True label — haqiqiy javob. Learn & Improve qismida model weightlarini yaxshilab, keyingi predictionda yaxshiroq natija chiqaradi.

Technical view — texnik ko'rinish. Bu yerda Deep Learning modelining ichki training jarayoni ko'rsatiladi. Jarayon quyidagicha ketadi:

Input Data → Model → Prediction → Loss →
Backpropagation → Optimizer Update → Improved Model

Input Data qismida images va labels bo'ladi. Images — rasmlar. Labels — haqiqiy classlar. Masalan, x — rasm, y — label.

Model qismida neural network inputni qayta ishlaydi.

Prediction qismida model output chiqaradi. Output matematik belgida ko'pincha $\hat{y}$ shaklida ko'rsatiladi. $\hat{y}$ — model predictioni. y — haqiqiy label.

Loss qismida model predictioni va true label orasidagi xato hisoblanadi. Masalan, Cross-Entropy Loss ishlatilishi mumkin. Cross-Entropy — classification vazifalarida keng ishlatiladigan loss function.

Backpropagation qismida gradientlar hisoblanadi. Gradientlar lossni kamaytirish uchun weightlar qaysi tomonga o'zgarishi kerakligini ko'rsatadi.

Optimizer Update qismida parameterlar yangilanadi. Bu matematik tarzda quyidagicha tasvirlanadi:

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \times \text{gradient}$$

θ — model parameterlari.

η — learning rate.

Gradient — lossni kamaytirish yo'nalishi.

Improved Model qismida model keyingi iterationda yaxshiroq natija chiqarishga tayyor bo'ladi. Iteration — bitta training qadami yoki bitta batch bo'yicha bajarilgan yangilanish.

Technical view chizmasi forward pass, backprop va update yo'nalishlarini ko'rsatadi. Forward pass inputdan predictiongacha boradi. Backprop lossdan orqaga qarab gradientlarni hisoblaydi. Update esa optimizer orqali weightlarni yangilaydi. Jarayon loss past va accuracy yuqori bo'lguncha ko'p marta takrorlanadi.

Accuracy — model nechta javobni to'g'ri topganini ko'rsatadigan metric. Metric — model natijasini o'lchaydigan ko'rsatkich.

Forward Pass nima?

Forward Pass — input ma'lumot model ichidan oldinga qarab o'tib, prediction hosil qiladigan jarayon. Forward — oldinga, pass — o'tish degani.

Forward Pass umumiy ko'rinishi:

Input → Model → Output

Image Classificationda bu quyidagicha bo'ladi:

Image → Neural Network → Prediction

Forward Pass jarayonida rasm modelga kiradi. Model ichidagi qatlamlar rasmni qayta ishlaydi. Oxirida class score yoki probability chiqadi.

Forward Pass o'quvchining savolga javob berishiga o'xshaydi. O'quvchi avval savolni o'qiydi, keyin o'ylaydi, keyin javob beradi. Modelda ham input kiradi, qatlamlar processing qiladi, output chiqadi.

Processing — qayta ishlash.

Forward Pass bosqichlari:

1. Input image
2. Neural network layers
3. Model output raw scores / logits
4. Softmax orqali probabilities

5. Highest probability asosida prediction

Input image — datasetdan olingan rasm. Masalan, mushuk rasmi.

Neural network layers — model qatlamlari. Neural network ichida input layer, hidden layer va output layer bo‘ladi. Hidden layer — yashirin qatlam. Bu qatlamlarda feature extraction va processing bajariladi.

Raw scores yoki logits — modelning softmaxdan oldingi chiqish qiymatlari. Logit — ehtimollikka aylantirilmagan score qiymati.

Softmax — logits qiymatlarini probabilityga aylantiradi. Probability — ehtimollik.

Highest probability — eng katta ehtimollik. Model odatda eng katta probabilityga ega classni prediction sifatida oladi.

Forward Pass misolida mushuk rasmi input sifatida beriladi. Neural network qatlamlaridan o‘tgandan keyin model output sifatida quyidagi raw scorelarni chiqaradi:

Dog: 2.50

Cat: 1.20

Bird: -0.30

Bu qiymatlar hali probability emas. Softmaxdan keyin ular quyidagicha probabilityga aylanadi:

Dog: 0.73

Cat: 0.24

Bird: 0.03

Eng katta probability Dog classida bo'lsa ham, rasm mushuk bo'lishi mumkin. Bu holatda model prediction sifatida Dog chiqaradi va xato qiladi. Agar rasmda haqiqatan mushuk bo'lsa, loss keyingi bosqichda shu xatoni o'lchaydi.

Forward Passda muhim jihat shuki, hali weightlar yangilanmaydi. Model faqat mavjud weightlar bilan prediction qiladi. O'rganish keyingi bosqichlarda, ya'ni loss, backpropagation va optimizer step orqali sodir bo'ladi.

Forward Pass uchun PyTorch kod misoli:

```
with torch.no_grad():  
    outputs = model(images)
```

`torch.no_grad()` — gradient hisoblamasdan prediction qilish rejimi. Bu odatda inference yoki evaluationda ishlatiladi. Trainingda esa gradient kerak bo'lgani uchun `torch.no_grad()` ishlatilmaydi.

Forward Pass haqida asosiy fikrlar:

Input modelga kiradi

Har bir layer ma'lumotni qayta ishlaydi

Output class score yoki probability beradi

Weightlar bu bosqichda yangilanmaydi

Learning keyingi backward passda boshlanadi

Layer — qatlam. Har bir layer inputni boshqa ko'rinishga o'tkazadi va foydali featurelarni ajratishga yordam beradi.

Forward Pass'ning simple view va technical view ko'rinishi

Forward Pass simple view ko'rinishida juda sodda tushuntiriladi. Rasm input sifatida keladi, CNN model uni tahlil qiladi, keyin predicted label chiqadi. Predicted label — model taxmin qilgan class nomi.

Masalan, tog' manzarasi rasmi input sifatida olinadi. CNN model rasmni tahlil qiladi. Yakuniy natijada model Mountain deb prediction qiladi. Confidence 98.2% bo'lishi mumkin. Confidence — modelning ishonch darajasi.

Simple view oqimi:

Input image → CNN model → Predicted label

CNN model — Convolutional Neural Network. O'zbekchada "konvolyutsion neyron tarmoq". CNN rasm ichidagi qirra, chiziq, shakl, rang va obyekt qismlarini ajratishda ishlatiladi.

Technical view ko'rinishida esa forward pass batafsilroq ko'rsatiladi:

Input image
↓
Convolution layers
↓
Pooling layers
↓
Fully connected layer
↓
Softmax output

Input image raw pixel qiymatlaridan iborat bo‘ladi. Masalan, $224 \times 224 \times 3$ o‘lchamdagi rasm. Bu rasm 224 pixel balandlik, 224 pixel kenglik va 3 ta RGB kanalga ega ekanini bildiradi.

Convolution layers filterlar yordamida feature extraction qiladi. Filter — rasm ustida yurib feature topadigan kichik weightlar oynasi. Masalan, filter qirra, chiziq yoki tekstura topishi mumkin. Texture — sirt naqshi yoki ko‘rinishi.

Pooling layers feature mapni downsample qiladi. Downsample — hajmni kichraytirish. Masalan, feature map $64 \times 112 \times 112$ dan $64 \times 56 \times 56$ ga tushishi mumkin. Bu muhim featurelarni saqlagan holda hisoblashni kamaytiradi.

Fully connected layer featurelarni birlashtirib, prediction uchun tayyorlaydi. Masalan, 512 birlikli fully connected layer ishlatilishi mumkin.

Softmax output class probabilitylarni chiqaradi. Misol sifatida quyidagi probabilitylar beriladi:

Mountain: 0.982

Forest: 0.012

Beach: 0.004

City: 0.001

Desert: 0.001

Probabilitylar yig‘indisi 1.0 bo‘ladi. Model eng katta probability bo‘lgan Mountain classini tanlaydi. Mountain — tog‘. Forest — o‘rmon. Beach — plyaj. City — shahar. Desert — cho‘l.

Forward Pass matematik jihatdan quyidagicha tasvirlanishi mumkin:

$$y = \text{softmax}(W_2 \cdot \sigma(W_1 \cdot \text{flatten}(\text{pool}(\text{conv}(X))))))$$

Bu formulada:

X — input image.

conv — convolution operatsiyasi.

pool — pooling operatsiyasi.

flatten — ko‘p o‘lchamli feature mapni bitta vectorga aylantirish.

W_1 va W_2 — weightlar.

σ — activation function, masalan ReLU.

softmax — probabilityga aylantirish.

Bu formula rasm convolution, pooling, flatten, fully connected va softmax orqali predictionga aylanishini ko‘rsatadi.

Loss nima?

Loss — model predictioni haqiqiy javobdan qanchalik uzoq ekanini o‘lchaydigan son. Loss model qanchalik noto‘g‘ri yoki yomon prediction qilganini bildiradi. Loss qancha katta bo‘lsa, xatolik shuncha katta bo‘ladi. Loss qancha kichik bo‘lsa, model predictioni shuncha yaxshi bo‘ladi.

Loss function — lossni hisoblaydigan funksiya. Image Classificationda Cross-Entropy Loss juda ko‘p ishlatiladi. Cross-Entropy Loss model haqiqiy classga qanchalik probability berganiga qarab xatoni hisoblaydi.

Loss imtihon bahosi analogiyasi orqali tushuntiriladi. Masalan, o‘quvchi 100 ballik imtihonda 92 ball oldi. Xato:

$$100 - 92 = 8 \text{ ball}$$

Bu yerda 8 ball xato miqdorini bildiradi. Modelda ham prediction haqiqiy javobdan qanchalik uzoq bo'lsa, loss shunchalik katta bo'ladi.

GPS analogiyasida ham loss tushuniladi. GPS manzilgacha bo'lgan masofani ko'rsatadi. Masofa qancha katta bo'lsa, xatolik shunchalik katta. Modelda loss ham prediction va haqiqiy label orasidagi "masofa"ga o'xshaydi.

Loss hisoblash misolida haqiqiy label Dog bo'ladi. Model prediction probabilities quyidagicha beriladi:

Dog: 0.30

Cat: 0.65

Bird: 0.05

True label — haqiqiy class. True label Dog. Model esa Cat classiga 0.65, Dog classiga 0.30 probability bergan. Bu modelning noto'g'ri tomonga ko'proq ishonganini bildiradi.

Loss qiymati misolda 0.65 sifatida yaqinlashtirilgan qiymat ko'rinishida ishlatiladi. Real Cross-Entropy Lossda esa haqiqiy class probabilitysi asosiy rol o'ynaydi.

Prediction output — model kiritilgan ma'lumot uchun ehtimollarni chiqaradi.

True Label — haqiqiy javob yoki to'g'ri class.

Loss — model predictioni qanchalik noto'g'ri ekanini o'lchaydi.

Interpretation — loss kichik bo'lsa, model yaxshiroq. Loss katta bo'lsa, model yomonroq.

High Loss va Low Loss misoli it rasmi orqali tushuntiriladi. High Loss holatida model true label Dog bo'lsa ham, Cat classiga yuqori ehtimollik beradi:

Dog: 0.05

Cat: 0.85

Bird: 0.10

Loss: 0.95

Bu katta xatolik. Chunki model haqiqiy dog classiga juda past ishonch bergan, noto'g'ri cat classiga esa juda yuqori ishonch bergan.

Low Loss holatida model haqiqiy classga yuqori ehtimollik beradi:

Dog: 0.92

Cat: 0.06

Bird: 0.02

Loss: 0.08

Bu kichik xatolik. Chunki model haqiqiy dog classiga 0.92 probability bergan.

Kodda loss quyidagicha hisoblanadi:

```
loss = criterion(outputs, labels)
```

Bu yerda `loss` — xato qiymati. `criterion` — loss function, masalan `CrossEntropyLoss`. `outputs` — model chiqishi. `labels` — haqiqiy javoblar.

Lossni qanday hisoblash

Lossni tushunish uchun bir nechta hisoblash usuli beriladi.

Birinchi usul — Wrong-class confidence. Wrong-class confidence — noto‘g‘ri classlar orasidagi eng katta probability. Wrong class — noto‘g‘ri class. Confidence — ishonch darajasi.

Misolda probabilitylar:

Dog: 0.30

Cat: 0.65

Bird: 0.05

Agar true class Dog bo‘lsa, wrong classes Cat va Bird bo‘ladi. Ularning probabilitylari 0.65 va 0.05. Eng kattasi:

$$\max(0.65, 0.05) = 0.65$$

Natija 0.65. Bu model noto‘g‘ri classga 65% ishonch berganini bildiradi.

Ikkinchi usul — Simple true-label error. Bu haqiqiy class probabilitysidan oddiy xato hisoblash usuli.

Formula:

$$1 - P(\text{true class})$$

$P(\text{true class})$ — haqiqiy class ehtimolligi. Agar true class Dog va $P(\text{Dog}) = 0.30$ bo‘lsa:

$$1 - 0.30 = 0.70$$

Natija 0.70. Bu oddiy xato ko‘rsatkichi bo‘lib, model haqiqiy classga yetarli ishonch bermaganini ko‘rsatadi.

Uchinchi usul — Real Cross-Entropy Loss. Classification vazifalarida real trainingda ko‘proq Cross-Entropy Loss ishlatiladi.

Formula:

$$\text{Loss} = -\log(P(\text{true class}))$$

Agar $P(\text{true class}) = 0.30$ bo‘lsa:

$$\text{Loss} = -\log(0.30) \approx 1.20$$

Natija 1.20 bo‘ladi.

Log — logarifm. Logarifm ehtimollik kichik bo‘lsa lossni katta qiladi, ehtimollik yuqori bo‘lsa lossni kichik qiladi.

Cross-Entropy Loss mantiqi:

$P(\text{true class})$ yuqori $\rightarrow$ loss kichik

$P(\text{true class})$ past $\rightarrow$ loss katta

Misollar:

$P(\text{true class}) = 0.90 \rightarrow$ loss kichik

$P(\text{true class}) = 0.30 \rightarrow$ loss kattaroq

$P(\text{true class}) = 0.01 \rightarrow$ loss juda katta

Lossni hisoblash jadvali uch xil qarashni ko‘rsatadi:

Wrong-class confidence $\rightarrow 0.65$

Simple true-label error $\rightarrow 0.70$

Real Cross-Entropy Loss $\rightarrow 1.20$

Bu uchala yondashuv modelning xatosini turli ko‘rinishda tushuntiradi. Amaliy Deep Learning trainingda Cross-Entropy Loss asosiy ishlatiladi.

Loss’ning simple view va technical view ko‘rinishi

Loss simple view ko‘rinishida nishonga o‘q otish bilan tushuntiriladi. Agar o‘q nishon markazidan uzoqqa tegsa, error katta bo‘ladi. Error — xato. Masofa qanchalik uzoq bo‘lsa, loss shunchalik katta bo‘ladi. Agar o‘q markazga yaqin tegsa, loss kichik bo‘ladi.

Image Classification misolida model Cat deb prediction qiladi, lekin ground truth Dog bo‘ladi. Ground truth — haqiqiy javob, true label bilan bir xil ma’noda ishlatiladi. Bu holatda prediction va haqiqiy javob mos kelmaydi. Shuning uchun error bor.

Simple view xulosasi:

Loss = prediction qanchalik noto‘g‘ri ekanini o‘lchaydi

Lower loss = better prediction

Lower loss — past loss. Better prediction — yaxshiroq prediction.

Technical view ko‘rinishida loss predicted probabilities va true label orasidagi farqni hisoblaydi. Predicted probabilities — model chiqargan class ehtimolliklari. True label one-hot ko‘rinishda beriladi. One-hot — faqat to‘g‘ri class 1, qolganlari 0 bo‘ladigan label formati.

Misol:

Predicted probabilities:

Cat: 0.10

Dog: 0.20

Bird: 0.70

True label one-hot:

Cat: 0

Dog: 0

Bird: 1

Bu holatda true class **Bird**, model esa Bird uchun 0.70 probability bergan. Cross-Entropy Loss:

$$\text{Loss} = -\log(0.70) \approx 0.357$$

Loss 0.357 nisbatan pastroq, chunki model true classga 0.70 ehtimollik bergan. Agar true class probability 0.10 bo'lganida, loss ancha kattaroq bo'lardi.

Loss grafik indikatorida “lower is better” ma'nosi ko'rsatiladi. Lower is better — pastroq qiymat yaxshiroq degani. Modelning maqsadi lossni minimallashtirish. Minimize — kamaytirish yoki eng kichik holatga olib kelish.

Backpropagation nima?

Backpropagation — model xatosini orqaga tarqatish va har bir weight uchun gradient hisoblash jarayoni. Backpropagation so'zini o'zbekchada “orqaga tarqatish” deb tushunish mumkin. Back — orqaga, propagation — tarqalish.

Forward passda input oldinga qarab harakat qiladi. Rasm input layerdan kiradi, hidden layerlardan o'tadi, output layerda prediction hosil bo'ladi. Loss hisoblangandan keyin backpropagation boshlanadi. Bu safar xato output layerdan oldingi qatlamlarga qarab qaytadi.

Backpropagation ta'rifi:

Backpropagation – loss qiymatidan foydalanib, har bir weight uchun gradientlarni hisoblash jarayoni

Gradient — lossni kamaytirish uchun weight qanday yo‘nalishda o‘zgarishi kerakligini ko‘rsatadigan qiymat.

Backpropagation o‘qituvchi va o‘quvchi analogiyasi bilan tushuntiriladi. O‘qituvchi o‘quvchining xatosini ko‘radi, qaysi qismda xato bo‘lganini ko‘rsatadi, keyin orqaga qarab tekshiradi va tuzatish yo‘lini aytadi. Modelda ham loss xato manbasini topishga yordam beradi.

Backpropagation muhimligi shundaki, har bir weightga “qancha va qaysi tomonga o‘zgarish kerak” degan signal beradi. Agar bu signal bo‘lmasa, model qaysi weightni qanday yangilashni bilmaydi.

Backpropagation chizmasida forward pass ko‘k o‘q bilan oldinga qarab ko‘rsatiladi. Input layerdan hidden layerlarga, hidden layerlardan output layerga ma’lumot oqadi. Prediction chiqadi. Keyin error/loss hosil bo‘ladi. Backward pass qizil o‘q bilan orqaga qarab ko‘rsatiladi. Error signal output layerdan oldingi hidden layerlarga va input tomonga qaytadi.

Input Layer — modelga featurelar kiradigan qatlam. Hidden Layers — model ichidagi yashirin qatlamlar. Output Layer — prediction chiqadigan qatlam.

Chizmada W^1 , W^2 , W^L kabi weightlar ko‘rsatiladi. Har bir weight uchun lossga nisbatan hosila hisoblanadi:

$$\partial L / \partial W$$

Bu ifoda “loss weightga nisbatan qanday o‘zgaradi?” degan ma’noni bildiradi.

$\partial L / \partial W$ katta bo'lsa, shu weight lossga kuchli ta'sir qilishi mumkin.
Optimizer shu gradientdan foydalanib weightni yangilaydi.

Gradient nima degan savolga quyidagicha javob beriladi:

Gradient – lossning weightga nisbatan hosilasi

Gradient modelga og'irlikni qanchaga va qaysi yo'nalishda o'zgartirish kerakligini ko'rsatadi.

Kodda backpropagation bitta qatorda bajariladi:

```
loss.backward()
```

Keyin weightlar optimizer orqali yangilanadi:

```
optimizer.step()
```

`loss.backward()` — gradientlarni hisoblaydi.

`optimizer.step()` — weightlarni yangilaydi.

Backpropagation nima uchun muhim:

Xato manbasini tarmoq bo'ylab topib beradi

Har bir weight uchun aniq yo'l-yo'riq beradi

Model keyingi qadamda yaxshilanishi uchun asos yaratadi

Qisqa xulosa:

Backpropagation modelga “qayerni va qanday tuzatish kerak”ligini bildiradigan mexanizm

Forward pass javobni hisoblaydi. Backward pass xatoni orqaga tarqatib gradientlarni beradi. Keyin weightlar yangilanadi va model o'rganadi.

Backpropagation'ning simple view va technical view ko'rinishi

Backpropagation simple view ko'rinishida o'quvchi xatolarini orqadan tekshirishga o'xshaydi. O'quvchi masalani oxirigacha ishlab, natijada xato qiladi. O'qituvchi oxirgi natijadan boshlab, qaysi stepda xato ketganini tekshiradi. Step — qadam. Backpropagation ham oxirgi lossdan boshlab oldingi qatlamlarga qarab xatoni tekshiradi.

Simple viewda bir nechta step ko'rsatiladi:

Step 1 → Step 2 → Step 3 → Step 4 → Result/Error

Xato result qismida ko'rinadi, lekin tuzatish uchun oldingi steplarga qaytish kerak bo'ladi. Backpropagation aynan shu ishni bajaradi.

Technical viewda neural network qatlamlari ko'rsatiladi:

Input Layer → Hidden Layer 1 → Hidden Layer 2 →
Output Layer → Target

Output va target orasidagi farq error/lossni beradi. Target — haqiqiy javob. Loss error signalga aylanadi. Error signal orqaga qarab hidden layerlarga qaytadi. Shu jarayonda gradientlar hisoblanadi. Gradientlar yordamida weightlar update qilinadi.

Chizmada ikki xil yo'nalish ko'rsatiladi:

Backward pass: gradientlar orqaga oqadi

Update weights: weightlar errorni kamaytirish uchun

moslashtiriladi

Goal — outputdan kelgan feedback asosida weightlarni o'zgartirib errorni kamaytirish.

Goal — maqsad.

Feedback — qayta aloqa, xatodan kelgan signal.

Backpropagation bo'lmasa, neural network xato qilganini bilishi mumkin, lekin qaysi qatlam yoki qaysi weight xatoga ko'proq sabab bo'lganini aniqlay olmaydi. Shuning uchun backpropagation Deep Learning trainingining eng asosiy mexanizmlaridan biri hisoblanadi.

Optimizer nima?

Optimizer — modelning weight qiymatlarini lossni kamaytirish uchun yangilaydigan algoritm. Optimizer so'zini o'zbekchada "optimallashtiruvchi algoritm" deb tushunish mumkin.

Optimizer vazifasi:

Gradientlardan foydalanish

Weightlarni yangilash

Lossni kamaytirish

Model accuracy'sini oshirish

Accuracy — modelning to'g'ri javoblar ulushi.

Optimizer ishlash jarayoni uch qadamda ko'rsatiladi:

1. Weightlar yangilanishdan oldin
2. Optimizer update qadami bajaradi

3. Weightlar yangilanganidan keyin

Masalan, weightlar oldin quyidagicha bo'lishi mumkin:

0.20
-0.10
0.05
0.30
-0.40

Optimizer gradientlardan foydalanib weightlarni ozgina o'zgartiradi. Yangilangandan keyin qiymatlar quyidagicha bo'lishi mumkin:

0.18
-0.07
0.02
0.28
-0.38

Bu kichik o'zgarishlar modelni lossni kamaytiradigan tomonga olib boradi. Har bir batchda optimizer step bajariladi. Ko'p batch va ko'p epoch davomida model asta-sekin yaxshilanadi.

Optimizer real hayotdagi murabbiyga o'xshatiladi. Murabbiy o'yin natijasini ko'radi, xatolarni tahlil qiladi, strategiyani o'zgartiradi va keyingi o'yinda yaxshiroq natija uchun tuzatish qiladi. Optimizer ham loss va gradientlarga qarab weightlarni o'zgartiradi.

Yana bir analogiyada tog'dan pastga tushish ko'rsatiladi. Maqsad eng past nuqtaga yetish. Loss landscape — loss qiymatlari joylashgan matematik manzara. Landscape — manzara. Yuqori loss yomon, past

loss yaxshi. Optimizer har qadamda loss pastroq bo'lgan tomonga yurishga harakat qiladi.

Loss landscape chizmasida qizil nuqta yuqori loss, yashil nuqta past loss sifatida ko'rsatiladi. Optimizer qadamlar orqali yuqori lossdan past lossga yaqinlashadi.

Mashhur optimizerlar:

SGD

Adam

SGD — Stochastic Gradient Descent. Stochastic — tasodifiy yoki kichik batchlar asosida ishlash. Gradient Descent — gradient bo'yicha pastga tushish, lossni kamaytirish usuli. SGD har batch yoki sample bo'yicha gradientni hisoblab, oddiy qadam bilan weightlarni yangilaydi. U oddiy va tez, lekin ba'zan notekis yo'l bilan boradi.

Adam — Adaptive Moment Estimation. Adam gradientning o'rtacha qiymati va kvadrat qiymatlaridan foydalanib, moslashuvchan qadam bilan yangilaydi. Adaptive — moslashuvchan. Moment — gradientlarning harakat yo'nalishi haqidagi ma'lumot. Adam tezroq konvergensiya berishi mumkin. Convergence — model loss minimumga yaqinlashib, barqaror o'rganishi.

PyTorchda optimizer quyidagicha ishlatiladi:

```
optimizer = torch.optim.Adam(model.parameters(),  
lr=0.001)
```

`model.parameters()` — modelning weight va bias kabi o'rganiladigan parametrlarini beradi.

$lr=0.001$ — learning rate. Learning rate — update qadamining kattaligi.

Training qadamida optimizer ishlashi:

```
optimizer.zero_grad()  
loss.backward()  
optimizer.step()
```

`zero_grad()` — eski gradientlarni tozalaydi.

`backward()` — gradientlarni hisoblaydi.

`step()` — weightlarni yangilaydi.

Qisqacha optimizer xulosasi:

Optimizer gradientlarning yoʻnalishini kuzatadi
Weightlarni yangilaydi
Maqsad lossni kamaytirish va model aniqligini oshirish
Yaxshi optimizer tezroq oʻrganish va yaxshi generalization beradi

Generalization — modelning yangi data'da ham yaxshi ishlashi.

Optimizer'ning simple view va technical view ko'rinishi

Optimizer simple view ko'rinishida tog'dan vodiya tushayotgan odam bilan tushuntiriladi. Odam yuqori nuqtadan boshlaydi. Yuqori nuqta high loss, ya'ni katta xatolik sifatida tushuniladi. Eng past vodiya esa lowest loss, ya'ni eng kichik xatolik. Odam kichik qadamlar bilan pastga tushadi. Har qadam lossni kamaytirishga o'xshaydi.

Simple viewdagi ma'no:

Start at high point → high loss

Take small steps → reduce loss

Reach bottom of valley → lowest loss

Optimizer modelni eng yaxshi mumkin bo'lgan yechimga olib borishga harakat qiladi.

Technical viewda optimizer loss function minimumini topishga harakat qiladi. Loss landscape chizmasida parameter θ_1 va θ_2 bo'yicha loss qiymatlari ko'rsatiladi. θ — model parameterlari. Optimizer current parametersdan boshlaydi, gradientni hisoblaydi va parameterlarni yangilaydi.

Technical view oqimi:

Current parameters θ_t

↓

Compute gradient $\nabla L(\theta_t)$

↓

Update parameters θ_{t+1}

↓

Repeat until convergence

$\nabla L(\theta_t)$ — loss gradienti. Gradient loss qaysi tomonga kamayishini ko'rsatadi.

Convergence — trainingning barqarorlashib, loss minimumga yaqinlashishi.

Common optimizers qismida SGD va Adam ko'rsatiladi. SGD sodda va samarali. Adam esa tez va moslashuvchan ishlashi bilan ko'p ishlatiladi.

Optimizerning asosiy vazifasi model parameterlarini shunday o'zgartirishki, loss kamayib, model performance yaxshilanishi kerak.

Learning Rate nima?

Learning Rate — training vaqtida har bir update qadamining kattaligini belgilaydigan hyperparameter. Hyperparameter — trainingdan oldin qo'lda tanlanadigan sozlama. Learning Rate qisqacha lr deb yoziladi.

Learning Rate model data'dan o'rganayotganda har bir update step qanchalik katta bo'lishini nazorat qiladi. Update step — weightlarni yangilash qadami.

Learning Rate ta'rifi:

Learning rate model o'rganayotganda har bir update qadamining kattaligini boshqaradi

Learning Rate qadam uzunligiga o'xshaydi. Biror manzilga yurilayotgan bo'lsa, juda kichik qadam bilan yurish juda sekin bo'ladi. Juda katta qadam bilan yurish esa manzildan o'tib ketish yoki adashishga olib kelishi mumkin. To'g'ri qadam esa manzilga tez va barqaror yetkazadi.

Real-life analogy — Walking to a Destination:

Too small → tiny steps → very slow progress

Too big → giant steps → overshoot and wander

Just right → steady steps → reach the goal

efficiently

Too small — juda kichik. Progress — rivojlanish. Too big — juda katta. Overshoot — kerakli joydan o'tib ketish. Just right — aynan mos. Efficiently — samarali.

Loss Landscape chizmasida learning rate ta'siri uch rangda tushuntiriladi:

Kichik learning rate yo'li mayda qadamlar bilan pastga tushadi. Bu juda sekin convergence beradi. Convergence — loss minimumga yaqinlashish.

To'g'ri learning rate yo'li loss minimumiga tez va barqaror tushadi. Bu eng yaxshi holat.

Katta learning rate yo'li minimumdan sakrab o'tib ketadi, tebranadi yoki diverge bo'ladi. Diverge — model o'rganishi beqaror bo'lib, loss kamaymasdan ketishi.

Effects of Learning Rate jadvali:

Too Small:

Effect: Tiny updates, slow progress

Outcome: Very slow convergence

Too Large:

Effect: Huge updates, overshoot, unstable

Outcome: May diverge or oscillate

Good Range:

Effect: Balanced updates, stable progress

Outcome: Fast and stable convergence

Tiny updates — juda kichik yangilanishlar.

Huge updates — juda katta yangilanishlar.

Oscillate — tebranish, loss bir pasayib, bir koʻtarilib turishi.

Balanced updates — muvozanatli yangilanishlar.

Stable progress — barqaror rivojlanish.

PyTorch kod misoli:

```
optimizer = Adam(model.parameters(), lr=0.001)
```

Bu yerda $lr=0.001$ learning rate qiymatini bildiradi. lr har bir update step hajmini boshqaradi.

Learning Rate haqida asosiy xulosa:

Learning rate learning tezligi va barqarorligini boshqaradi

Juda kichik boʻlsa – juda sekin

Juda katta boʻlsa – beqaror

Toʻgʻri tanlansa – eng yaxshi natija

Learning Rate'ning simple view va technical view koʻrinishi

Learning Rate simple viewda maqsadga yetish uchun har xil step size bilan yurish sifatida tushuntiriladi. Step size — qadam kattaligi.

Juda kichik step size boʻlsa, odam maqsadga juda sekin yaqinlashadi. Har qadam kichik, progress sekin.

Yaxshi step size bo'lsa, odam barqaror qadamlar bilan maqsadga yetadi. Bu trainingda lossning tez va barqaror kamayishiga o'xshaydi.

Juda katta step size bo'lsa, odam maqsaddan oshib ketadi yoki maqsad atrofida aylanib yuradi. Bu trainingda loss minimumdan sakrab o'tib ketishiga o'xshaydi.

Technical viewda loss function grafigi U shaklida ko'rsatiladi. Y o'qi loss, X o'qi model parameters. Model parameterlari o'zgarganda loss ham o'zgaradi. Maqsad loss eng past joyga tushish.

Kichik learning rate chizig'i sekin-sekin minimumga boradi. Good learning rate chizig'i minimumga tezroq va samarali yaqinlashadi. Juda katta learning rate chizig'i esa minimumdan oshib ketadi va tebranadi.

Key idea:

Learning rate optimization paytida olinadigan step size'ni boshqaradi

Uni to'g'ri tanlash kerak – juda kichik ham emas, juda katta ham emas

Optimization — lossni kamaytirish jarayoni.

Epoch va Batch nima?

Epoch va Batch training jarayonining necha marta va qanday qismlarda bajarilishini tushuntiradi.

Batch — datasetdan bir marta olinadigan kichik qism. Model batch ustida o'rganadi va weightlar yangilanadi. Mini-batch — batchning odatiy ishlatiladigan kichik ko'rinishi. Ko'p traininglarda mini-batch gradient descent ishlatiladi.

Epoch — butun datasetdan bir marta to‘liq o‘tish. Barcha batchlar ketma-ket ko‘rilsa, bitta epoch tugaydi.

Batch ta’rifi:

Batch – datasetdan bir marta olinadigan kichik qism
Model bir batch ustida o‘rganadi va yangilanadi

Epoch ta’rifi:

Epoch – butun datasetdan bir marta to‘liq o‘tish
Barcha batchlar ketma-ket ko‘riladi

Real-life analogy sifatida kitob o‘qish misoli ishlatiladi. Kitobni butunlay o‘qib chiqish — bitta epoch. Har safar bir necha sahifa o‘qish — batch. Barcha sahifalar o‘qib tugatilsa, bitta epoch tugaydi.

Imtihon qog‘ozlari analogiyasida o‘qituvchi barcha qog‘ozlarni birdan tekshirmaydi, kichik guruhlariga bo‘lib tekshiradi. Har bir kichik guruh batch. Barcha guruhlar tekshirilgach, bitta epoch tugaydi.

Diagrammada 1000 ta rasmlı dataset misol qilib olingan. Batch size 100 bo‘lsa, dataset 10 ta batchga bo‘linadi:

Dataset = 1000 rasm

Batch size = 100

Batches per epoch = $1000 / 100 = 10$

1 epoch = 10 batch

Batchlar model orqali ketma-ket o‘tadi:

Batch 1 → Model → Output / Loss
Batch 2 → Model → Output / Loss
...
Batch 10 → Model → Output / Loss

Barcha 10 batch tugasa, 1 epoch tugaydi. Epoch counter qismida Epoch 1, Epoch 2, Epoch 3, ..., Epoch N ketma-ketligi ko'rsatiladi. Counter — sanagich.

Qisqa misol:

Dataset: 1000 ta rasm

Batch size: 100

Batches per epoch = $1000 / 100 = 10$

1 epoch = 10 ta batch

5 epoch uchun jami iteratsiyalar = $10 \times 5 = 50$

Iteration — bitta batch bo'yicha bajariladigan training qadami. Har iterationda forward pass, loss, backward pass va optimizer step bajariladi.

PyTorch kod namunasi:

```
for epoch in range(num_epochs):
    for images, labels in train_loader:
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
```

Bu yerda tashqi loop epochlar bo'yicha aylanadi. Ichki loop `train_loader`dagi batchlar bo'yicha aylanadi.

Atamalar jadvali:

Batch — datasetdan olingan kichik qism. Model bir marta ko'radigan namunalar guruhi. Bitta forward va backward qadamidan keyin tugaydi.

Mini-batch — batch size bilan olingan kichik guruh. Har bir iteratsiyada modelga beriladigan ma'lumot. Model parametrlari bir marta yangilanadi.

Iteration — bitta mini-batch ustida bitta o'qitish qadami. Forward + backward + update bajarilishi bilan tugaydi.

Epoch — butun datasetdan bir marta to'liq o'tish. Oxirgi batch ham ko'rilganda tugaydi.

Epoch va Batch'ning simple view va technical view ko'rinishi

Epoch and Batch simple viewda kitob o'qish va olma saralash misollari orqali tushuntiriladi.

Kitob o'qish misolida:

Epoch → butun kitobni bir marta o'qish

Batch → har safar bir necha sahifani o'qish

Kitob birdan to'liq o'qilmasligi mumkin. Har safar bir necha sahifa o'qiladi. Barcha sahifalar tugasa, kitob bir marta to'liq o'qilgan bo'ladi. Bu 1 epochga o'xshaydi.

Olma saralash misolida:

Epoch → barcha olmalarni bir marta saralash
Batch → har safar kichik olma guruhini saralash

Dataset ham shunday ishlaydi. Barcha rasmlar birdan modelga berilmaydi. Kichik batchlarga bo‘linadi. Model har batchdan o‘rganadi.

Technical viewda dataset barcha training samples sifatida ko‘rsatiladi. Dataset batch 1, batch 2, batch 3, ..., batch N ga bo‘linadi. Har batch modelga kiradi. Model bir vaqtning o‘zida bitta batch ustida train qilinadi. Barcha batchlar ko‘rilgach, 1 epoch completed bo‘ladi. Completed — yakunlangan.

Technical viewdagi asosiy fikr:

Batch = small step

Epoch = complete journey

Small step — kichik qadam. Complete journey — to‘liq yo‘l.

Training ko‘p epoch davom etadi. Har epochda model butun datasetni yana bir marta ko‘radi. Shu sababli model ko‘proq mashq qiladi. Lekin epoch juda ko‘p bo‘lsa, overfitting xavfi ham oshadi.

Overfitting — model training data’ni yodlab olib, yangi data’da yomon ishlashi.

Praktika: One Training Step PyTorch

One Training Step — bitta batch ustida bajariladigan to‘liq training qadami. Bu qadam forward pass, loss, backward pass va optimizer stepdan iborat bo‘ladi.

Training loop mantiqi quyidagicha:

1. `images, labels = batch` ma'lumotni olish
2. `outputs = model(images)` → Forward pass
3. `loss = criterion(outputs, labels)` → Loss hisoblash
4. `optimizer.zero_grad()` → Eski gradientlarni tozalash
5. `loss.backward()` → Gradientlarni hisoblash
6. `optimizer.step()` → Weightlarni yangilash
7. Jarayon har batch uchun takrorlanadi
8. Barcha batchlar tugasa → 1 epoch bo'ladi

`images` — batchdagi rasmlar. `labels` — shu rasmlarning haqiqiy javoblari.

`outputs = model(images)` — model rasmlarni qabul qilib prediction chiqaradi. Bu forward pass.

`loss = criterion(outputs, labels)` — model predictioni true label bilan solishtirilib loss hisoblanadi.

`optimizer.zero_grad()` — eski gradientlarni tozalaydi.

PyTorchda gradientlar avtomatik yig'ilib borishi mumkin. Shuning uchun yangi batchdan oldin gradientlarni tozalash kerak.

`loss.backward()` — backpropagation bajaradi, ya'ni gradientlarni hisoblaydi.

`optimizer.step()` — optimizer weightlarni yangilaydi.

PyTorch code:

```
for images, labels in train_loader:
    optimizer.zero_grad()
    outputs = model(images)           # forward pass
```



```
    loss = criterion(outputs, labels)
    loss.backward()
backpropagation
    optimizer.step()
weights
```

`train_loader` — batchlarni ketma-ket beradigan PyTorch `DataLoader`. `DataLoader` — datasetdan batch tayyorlab beradigan vosita.

Forward pass va backward pass birgalikda training stepni tashkil qiladi. Mini xulosa:

Training loop = Forward → Loss → Backward → Step → Repeat

Forward — model prediction chiqaradi.

Loss — prediction xatosi hisoblanadi.

Backward — gradientlar hisoblanadi.

Step — weightlar yangilanadi.

Repeat — jarayon takrorlanadi.

Bu kod Deep Learning trainingining eng asosiy skeleti hisoblanadi. Model har batchda shunday o'rganadi.

Training & Optimization bo'yicha muhim eslatmalar

Training jarayonida bir nechta muhim holatlarni doim kuzatib borish kerak.

Learning rate juda kichik bo'lsa, training juda sekin ketadi. Model lossni kamaytiradi, lekin juda uzoq vaqt talab qiladi. Loss curve juda sekin pastga tushadi.

Learning rate juda katta bo'lsa, model tebranadi yoki diverge bo'ladi. Diverge — loss kamaymay, training beqaror bo'lib ketishi. Model minimumga tushish o'rniga undan sakrab o'tib ketadi.

Loss pasaymayotgan bo'lsa, data, model yoki optimizer sozlamalarini tekshirish kerak. Data noto'g'ri bo'lishi, label xato bo'lishi, preprocessing noto'g'ri bajarilishi, model juda sodda yoki learning rate noto'g'ri bo'lishi mumkin.

Overfitting holatida train yaxshi, validation yomon bo'ladi. Train accuracy yuqori, validation accuracy past bo'lsa, model training data'ni yodlab olgan bo'lishi mumkin. Bunday holatda augmentation yoki regularization kerak bo'ladi.

Augmentation — rasmlarni turli ko'rinishga keltirib, datasetni boyitish. Masalan, rasmni aylantirish, flip qilish, brightness o'zgartirish.

Regularization — overfittingni kamaytirish usullari. Masalan, dropout, weight decay, early stopping.

Dropout — training paytida ayrim neuronlarni vaqtincha o'chirib turish.

Weight decay — weightlarni haddan tashqari katta bo'lib ketishdan saqlash.

Early stopping — validation natija yomonlashsa trainingni to'xtatish.

Batch size ta'siri ham muhim. Katta batch stability beradi. Stability — barqarorlik. Kichik batch esa ba'zan yaxshiroq generalization beradi. Generalization — modelning yangi data'da yaxshi ishlashi.

Monitoring — training jarayonini kuzatish. Loss curve va accuracy har epoch kuzatib boriladi. Loss curve — lossning vaqt bo'yicha o'zgarish grafigi. Accuracy — to'g'ri javoblar ulushi.

Kuzatiladigan asosiy qiymatlar:

Train loss

Validation loss

Train accuracy

Validation accuracy

Learning rate

Epoch natijalari

Agar train loss kamayib, validation loss oshsa, overfitting bo'lishi mumkin. Agar ikkala loss ham kamaymasa, model o'rganmayotgan bo'lishi mumkin. Agar loss juda sakrab tursa, learning rate katta bo'lishi mumkin.

Summary: asosiy terminlar

Training — modelni data orqali o'qitish jarayoni. Model rasmlar va labellar asosida to'g'ri prediction qilishni o'rganadi.

Optimization — lossni kamaytirish uchun parameterlarni yangilash. Model xatoni kamaytirish uchun weight va bias qiymatlarini o'zgartiradi.

Forward Pass — input modeldan o'tib output yoki prediction beradigan jarayon. Bu bosqichda model prediction chiqaradi, lekin hali o'rganmaydi.

Loss — prediction qanchalik xato ekanini o'lchaydi. Loss katta bo'lsa, model yomonroq prediction qilgan. Loss kichik bo'lsa, prediction yaxshiroq.

Backpropagation — xatoni orqaga tarqatib gradientlarni hisoblaydi. Bu jarayon modelga qaysi weightlarni qanday o'zgartirish kerakligini bildiradi.

Optimizer — gradientlardan foydalanib weightlarni yangilaydi. Optimizer modelni loss kamayadigan yo'nalishda harakatlantiradi.

Learning Rate — har update qadamining kattaligini belgilaydi. Juda kichik learning rate sekin o'rganishga, juda katta learning rate beqaror trainingga olib keladi.

Batch — datasetning kichik qismi. Model har batch ustida bitta training step bajaradi.

Epoch — barcha batchlarni bir marta to'liq ko'rib chiqish. 1 epoch tugashi uchun butun training dataset modeldan bir marta o'tishi kerak.

Eng muhim formula flow:

Input → Model → Prediction → Loss → Backpropagation
→ Optimizer Step → Better Model

Bu flow model o'rganishining asosiy yo'li hisoblanadi.

Architecture modelning tanasi bo'lsa, training va optimization uning o'rganish jarayonidir. Model architecture qurilgandan keyin u data orqali training qilinadi. Training davomida model prediction qiladi, loss hisoblaydi, backpropagation orqali xatoni orqaga tarqatadi, optimizer orqali weightlarni yangilaydi va asta-sekin yaxshilanadi.

Yakuniy tushuncha

Computer Vision modelini o'qitishda faqat model architecture yetarli bo'lmaydi. Architecture rasm qanday qatlamlardan o'tishini

belgilaydi, lekin modelning o'rganishi training and optimization orqali sodir bo'ladi.

Model avval forward pass orqali prediction chiqaradi. Prediction haqiqiy label bilan solishtiriladi. Loss model xatosini ko'rsatadi. Backpropagation lossdan gradientlarni hisoblaydi. Optimizer gradientlardan foydalanib weightlarni yangilaydi. Learning rate update qadamining kattaligini boshqaradi. Batchlar datasetni kichik qismlarga ajratadi. Epoch esa butun datasetni bir marta to'liq ko'rishni bildiradi.

Rasmli chizmalar asosida butun jarayon shunday ko'rinadi: rasm datasetdan olinadi, model ichiga kiradi, neural network prediction chiqaradi, loss xatoni o'lchaydi, backward pass orqali xato orqaga tarqaladi, optimizer weightlarni yangilaydi, model keyingi batchda yaxshiroq prediction qilishga harakat qiladi. Bu sikl ko'p batch va ko'p epoch davomida takrorlanganda model rasm ichidagi featurelarni yaxshiroq ajratadi va classlarni aniqroq topishni o'rganadi.